

算法设计与分析

Design & Analysis of Algorithms

第3章 动态规划

学习要点

- 理解动态规划算法的概念
- 掌握动态规划算法的基本要素
 - (1) 最优子结构性性质 (2) 重叠子问题性质
- 掌握设计动态规划算法的步骤
- 通过应用范例学习动态规划算法设计策略
 - (1) 基础题目
 - (2) 矩阵连乘问题
 - (3) 背包问题
 - (4) 子序列问题

3.5 子序列问题

- 子序列(不连续的)
 - 最长公共子序列
- 子序列(连续)
 - 最大子段和

3.5 子序列问题 最长公共子序列

- 子序列 将给定序列中零个或多个元素去掉后所得结果
- 示例

给定序列X

X

A	B	C	B	D	A	B
---	---	---	---	---	---	---

X的子序列

X_1

A	B	C	B	D	A	B
---	---	---	---	---	---	---

X_2

A	B	C	B
---	---	---	---

X_3

A	C	B	B
---	---	---	---

3.5 子序列问题 最长公共子序列

- 给定两个序列X和Y

X

A	B	C	B	D	A	B
---	---	---	---	---	---	---

Y

B	D	C	A	B	A
---	---	---	---	---	---

- 公共子序列

X_1

C	A
---	---

Y_1

C	A
---	---

X_2

A	B	A
---	---	---

Y_2

A	B	A
---	---	---

X_3

B	C	A	B
---	---	---	---

Y_3

B	C	A	B
---	---	---	---

- 定义

输入：序列 $X = \langle x_1, x_2, \dots, x_n \rangle$ 和序列 $Y = \langle y_1, y_2, \dots, y_m \rangle$

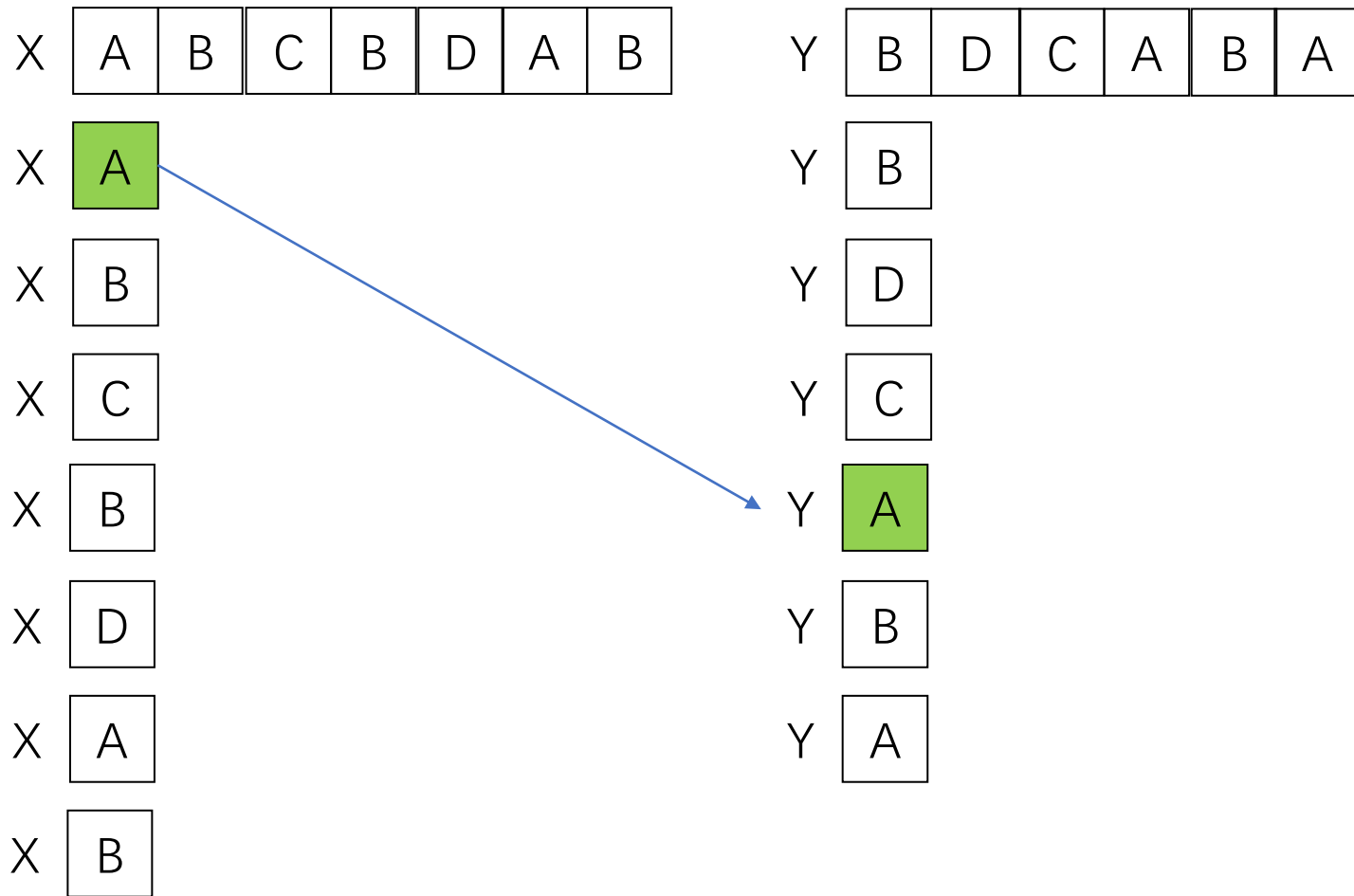
输出：求解一个公共子序列 $Z = \langle z_1, z_2, \dots, z_p \rangle$ ，令 $\max|Z|$

满足 $\langle z_1, z_2, \dots, z_p \rangle = \langle x_{i_1}, x_{i_2}, \dots, x_{i_p} \rangle = \langle y_{j_1}, y_{j_2}, \dots, y_{j_p} \rangle$

$(1 \leq i_1 < i_2, \dots, i_p \leq n; 1 \leq j_1 < j_2, \dots, j_p \leq m)$

3.5 子序列问题 最长公共子序列

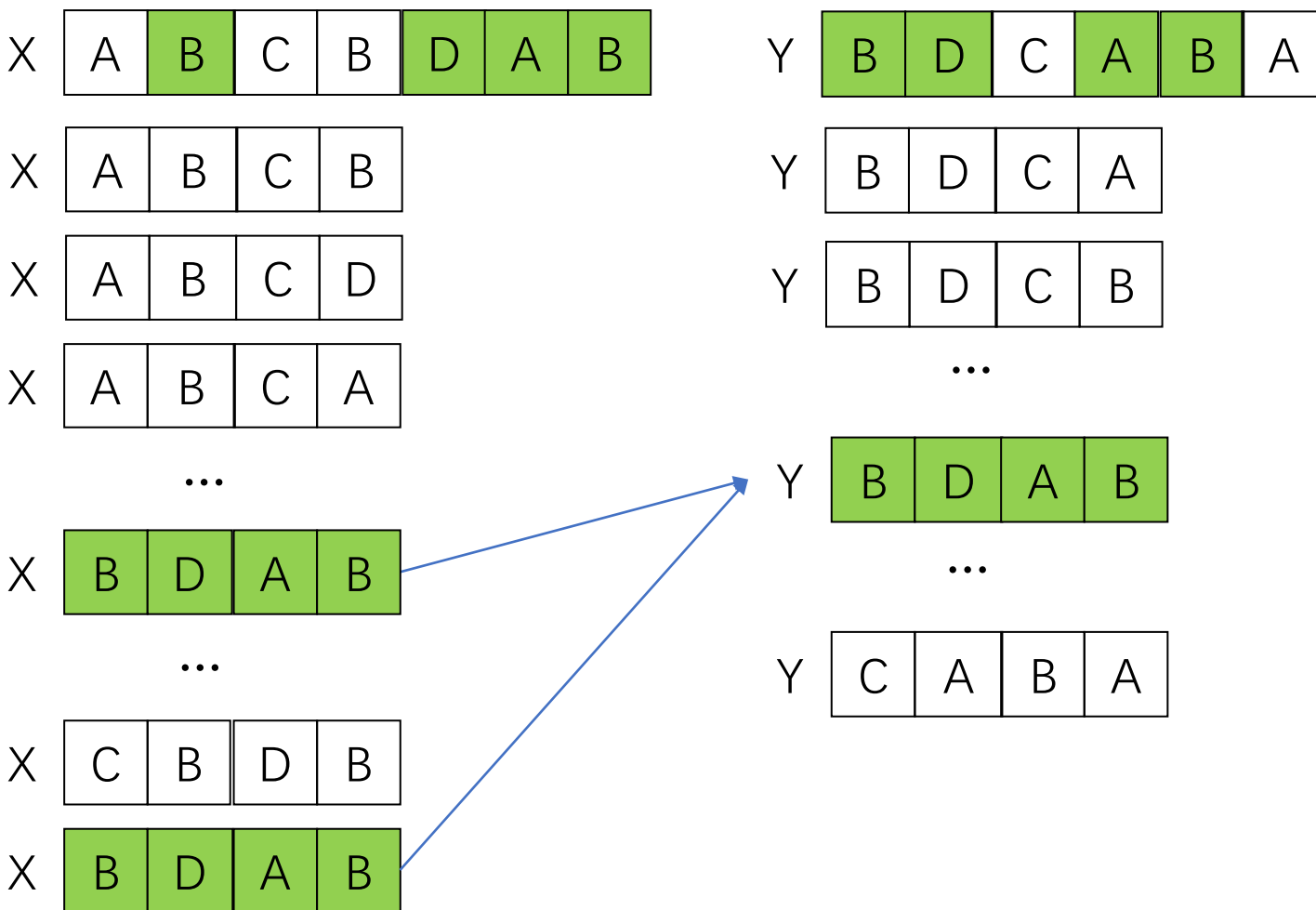
- 蛮力枚举 枚举所有子序列 长度从1, 2, 3, ...



枚举并检查长度为1的子序列

3.5 子序列问题 最长公共子序列

- 蛮力枚举 枚举所有子序列 长度从1, 2, 3, ...



枚举并检查长度为4的子序列

3.5 子序列问题 最长公共子序列

- 蛮力枚举 枚举所有子序列 长度从1, 2, 3, ...

X A B C B D A B

Y B D C A B A

X B

Y B

长度为1

X B D

Y B D

长度为2

X B D A

Y B D A

长度为3

X B D A B

Y B D A B

长度为4

最长公共子序列

3.5 子序列问题 最长公共子序列

- 枚举观察

X

B	D	A	B
---	---	---	---

Y

B	D	A	B
---	---	---	---

长度为4

X

D	A	B
---	---	---

Y

D	A	B
---	---	---

长度为3

存在最优子结构和重叠子问题

3.5 子序列问题 最长公共子序列

■ 枚举观察

X

B	D	A	B
---	---	---	---

Y

B	D	A	B
---	---	---	---

长度为4

X

D	A	B
---	---	---

Y

D	A	B
---	---	---

长度为3

X

A	B
---	---

Y

A	B
---	---

长度为2

存在最优子结构和重叠子问题

3.5 子序列问题 最长公共子序列

■ 枚举观察

X B D A B

Y B D A B

长度为4

X D A B

Y D A B

长度为3

X A B

Y A B

长度为2

X B

Y B

长度为1

存在最优子结构和重叠子问题

3.5 子序列问题 最长公共子序列

- 给出问题表示

$c[i,j]$: $X[1...i]$ 和 $Y[1...j]$ 的最长公共子序列长度

X	x_1	x_2	$\cdots$	x_{i-1}	x_i
Y	y_1	y_2	$\cdots$	y_{i-1}	y_j

- 明确原始问题

$c[n,m]$: $X[1...n]$ 和 $Y[1...m]$ 的最长公共子序列长度

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.5 子序列问题 最长公共子序列

■ 考察末尾字符

情况1: $x_7 \neq y_6$

X A B C B D A **B**

Y B D C A B **A**

情况2: $x_7 = y_5$

X A B C B D A **B**

Y B D C A **B**

问题结构分析

递推关系建立

自底向上计算

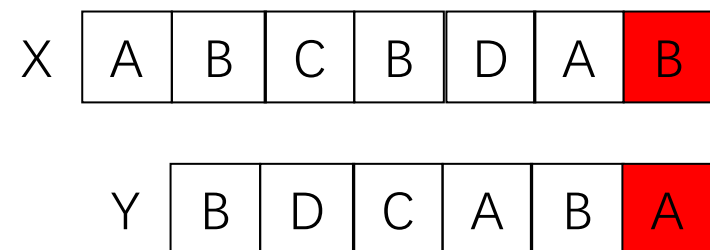
最优方案追踪

3.5 子序列问题 最长公共子序列

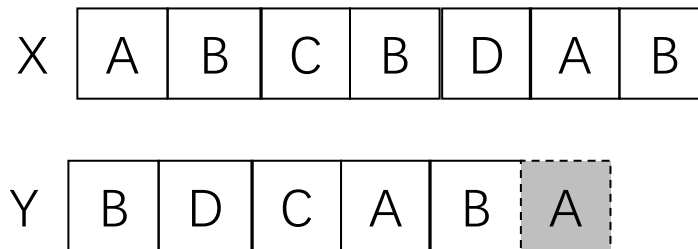
■ 考察末尾字符

情况1: $x_7 \neq y_6$

$c[7,6]$

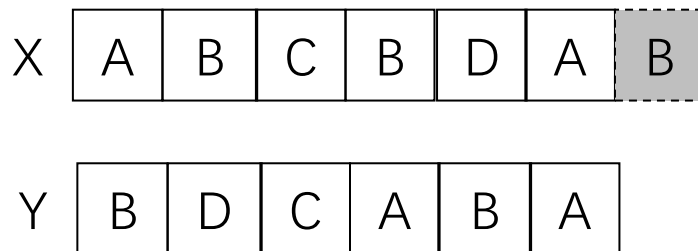


$c[7,6-1]+0$



max

$c[7-1,6]+0$



情况1: $x_i \neq y_j$

$$c[i,j] = \max\{c[i-1,j], c[i,j-1]\}$$

最优子结构

问题结构分析

递推关系建立

自底向上计算

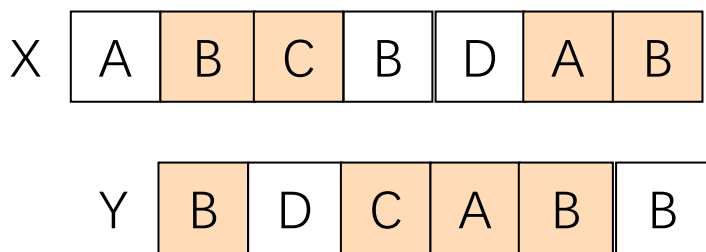
最优方案追踪

3.5 子序列问题 最长公共子序列

- 考察末尾字符

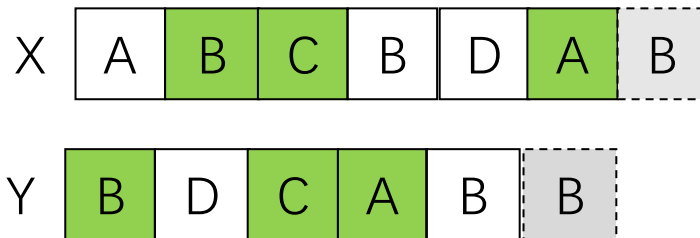
情况2: $x_7 = y_6$

$c[7,6]$

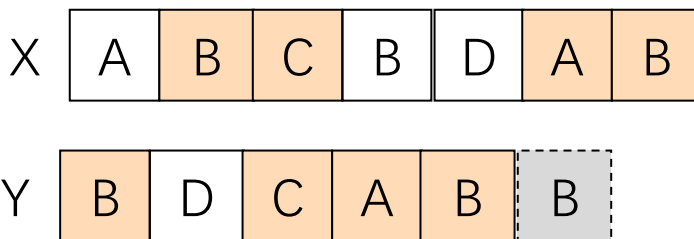


也可不同时出现

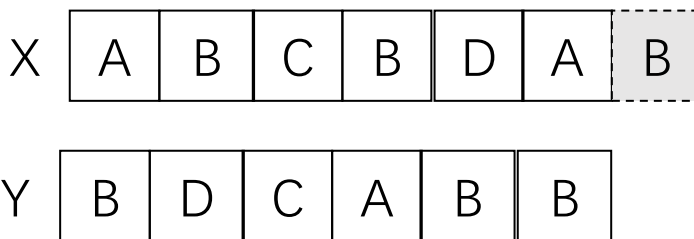
$c[7-1,6-1]+1$



$c[7,6-1]+0$



$c[7-1,6]+0$



问题结构分析

递推关系建立

自底向上计算

最优方案追踪

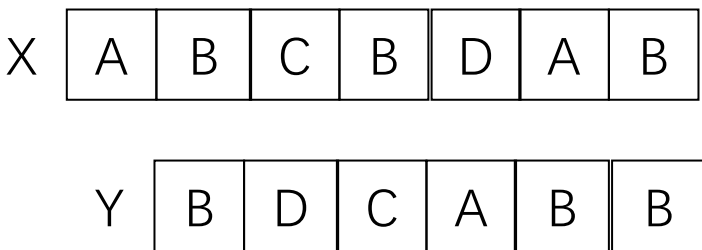
3.5 子序列问题 最长公共子序列

- 考察末尾字符

情况2: $x_7 = y_6$

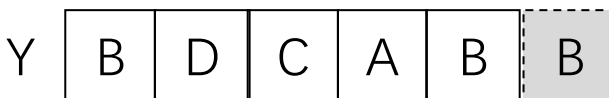
$c[7,6]$

max

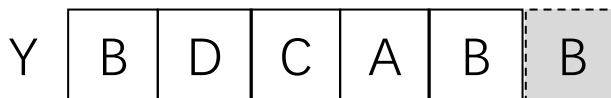
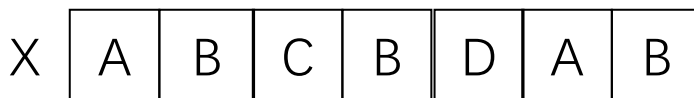


也可不同时出现

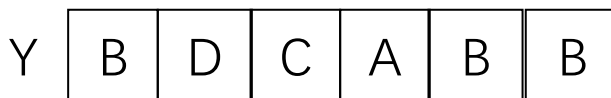
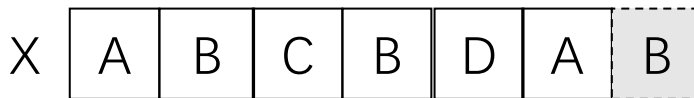
$c[7-1,6-1]+1$



$c[7,6-1]+0$



$c[7-1,6]+0$



问题结构分析

递推关系建立

自底向上计算

最优方案追踪

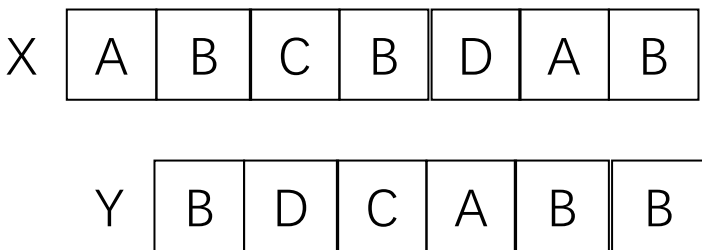
3.5 子序列问题 最长公共子序列

■ 考察末尾字符

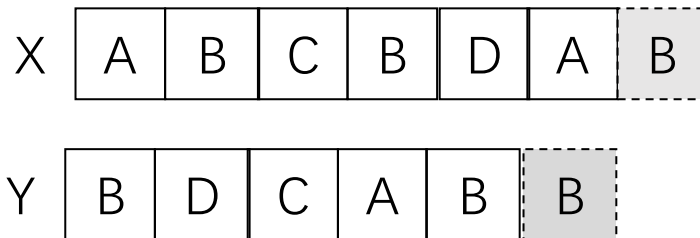
情况2: $x_i = y_j$

$c[i,j]$

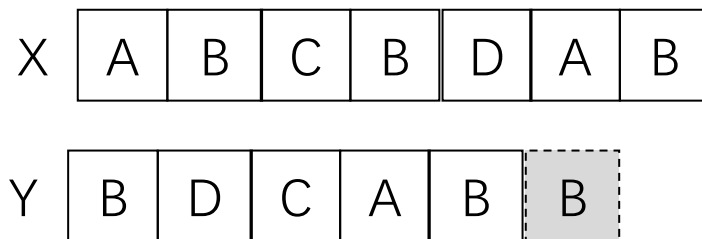
max



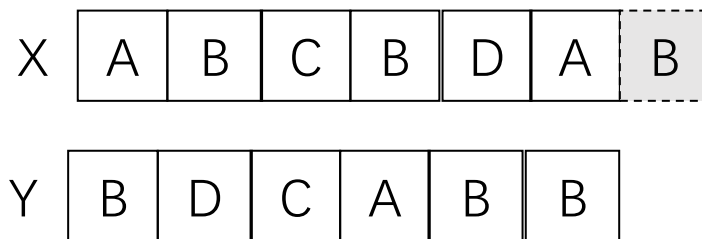
$c[i-1,j-1]+1$



$c[i,j-1]+0$



$c[i-1,j]+0$



问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.5 子序列问题 最长公共子序列

■ $x_i = y_j$

$c[i,j-1]$ 比 $c[i-1,j-1]$ 至多大1

$c[i-1,j]$ 比 $c[i-1,j-1]$ 至多大1

$c[i-1,j-1]+1$, 另外两个+0

$\max$
 $c[i,j]$

X A B C B D A x_i

Y B D C A B y_j

$c[i-1,j-1]+1$

$\geq \max\{c[i,j-1], c[i-1,j]\}$

$c[i,j] = c[i-1,j-1] + 1$

$c[i-1,j-1]+1$ 已充分

X A B C B D A B

Y B D C A B B

$c[i,j-1]+0$ 非必要

X A B C B D A B

Y B D C A B B

$c[i-1,j]+0$ 非必要

X A B C B D A B

Y B D C A B B

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

3.5 子序列问题 最长公共子序列

■ 递推公式

$$c[i,j] = \begin{cases} \max\{c[i,j-1], c[i-1,j]\}, & x_i \neq y_j \\ c[i-1,j-1] + 1, & x_i = y_j \end{cases}$$

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

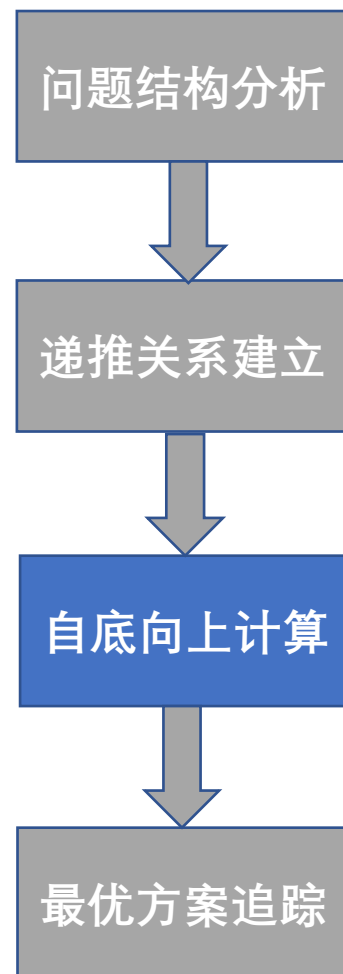
3.5 子序列问题 最长公共子序列

■ 初始化

$$c[i,0]=c[0,j]=0$$

序列长度为0时，最长公共子序列长度为0

$c[i,j]$	$j=0$	$j=1$	$j=2$	$\dots$	$j=m$
$i=0$	0	0	0	0	0
$i=1$	0				
$i=2$	0				
$\dots$	0				
$i=n$	0				



3.5 子序列问题 最长公共子序列

■ 初始化

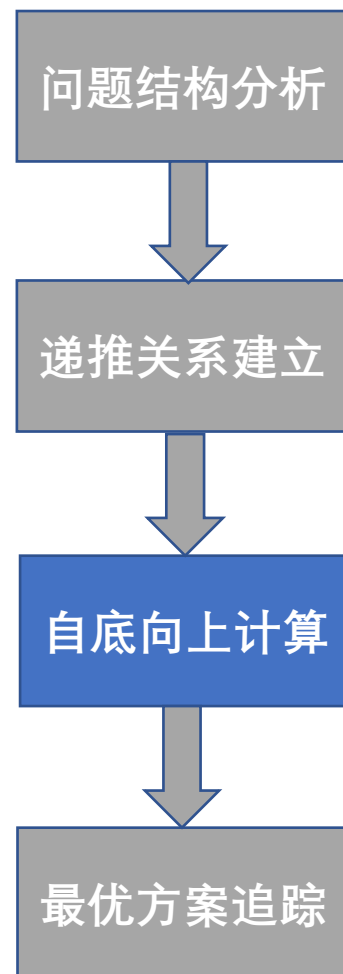
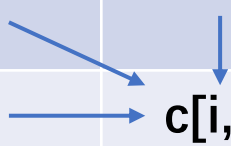
$$c[i,0]=c[0,j]=0$$

序列长度为0时，最长公共子序列长度为0

■ 递推公式

$$c[i,j]=\begin{cases} \max\{c[i,j-1], c[i-1,j]\}, & x_i \neq y_j \\ c[i,j] = c[i-1,j-1] + 1, & x_i = y_j \end{cases}$$

$c[i,j]$	$j=0$	$j=1$	$j=2$	$\dots$	$j=m$
$i=0$	0	0	0	0	0
$i=1$	0				
$i=2$	0				
$\dots$	0				
$i=n$	0				



3.5 子序列问题 最长公共子序列

■ 初始化

$$c[i,0]=c[0,j]=0$$

序列长度为0时，最长公共子序列长度为0

■ 递推公式

$$c[i,j]=\begin{cases} \max\{c[i,j-1], c[i-1,j]\}, & x_i \neq y_j \\ c[i-1,j-1]+1, & x_i = y_j \end{cases}$$

$c[i,j]$	$j=0$	$j=1$	$j=2$	$\dots$	$j=m$
$i=0$	0	0	0	0	0
$i=1$	0				
$i=2$	0				
$\dots$	0				
$i=n$	0				

问题结构分析

递推关系建立

自底向上计算

最优方案追踪

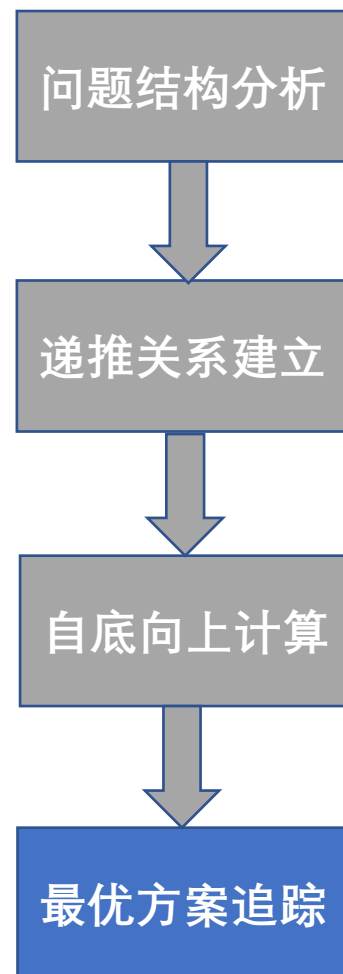
3.5 子序列问题 最长公共子序列

- 构造追踪数组 $rec[1...n]$, 记录子问题来源枚举观察

$$rec[i,j] = \begin{cases} LU, & \text{if } c[i,j] = c[i-1,j-1] + 1 \\ U, & \text{if } c[i,j] = c[i-1,j] \\ L & \text{if } c[i,j] = c[i,j-1] \end{cases}$$

$c[i,j]$	$j=0$	$j=1$	$j=2$	...	$j=m$
$i=0$	0	0	0	0	0
$i=1$	0			$rec[] = LU$	
$i=2$	0				
...	0				
$i=n$	0		$rec[] = U$		

$rec[] = L$



3.5 子序列问题 最长公共子序列

■ 算法实例

	1	2	3	4	5	6	7
x_i	A	B	C	B	D	A	B
y_j	B	D	C	A	B	A	

$x_i = y_j$

$c[]$

	j=0	1	2	3	4	5	6
i=0	0	0	0	0	0	0	0
1	0	0	0	0	1		
2							
3							
4							
5							
6							
7							

$C[1,4] = c[0,3] + 1$

$rec[]$

	j=1	2	3	4	5	6
i=1	U	U	U	LU		
2						
3						
4						
5						
6						
7						

3.5 子序列问题 最长公共子序列

■ 算法实例

	1	2	3	4	5	6	7
x_i	A	B	C	B	D	A	B
y_j	B	D	C	A	B	A	

$c[]$

	j=0	1	2	3	4	5	6
i=0	0	0	0	0	0	0	0
1	0	0	0	0	1		
2							
3							
4							
5							
6							
7							

$rec[]$

	j=1	2	3	4	5	6
i=1	U	U	U	LU	L	
2						
3						
4						
5						
6						
7						

3.5 子序列问题 最长公共子序列

■ 算法实例

	1	2	3	4	5	6	7
x_i	A	B	C	B	D	A	B
y_j	B	D	C	A	B	A	

c[]

	j=0	1	2	3	4	5	6
i=0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	1	1	1	1	2	2
3	0	1	1	2	2	2	2
4	0	1	1	2	2	3	3
5	0	1	2	2	2	3	3
6	0	1	2	2	3	3	4
7	0	1	2	2	3	4	?

rec[]

	j=1	2	3	4	5	6
i=1	U	U	U	LU	L	LU
2	LU	L	L	U	LU	L
3	U	U	LU	L	U	U
4	LU	U	U	U	LU	L
5	U	LU	U	U	U	U
6	U	U	U	LU	U	LU
7	LU	U	U	U	LU	?

3.5 子序列问题 最长公共子序列

■ 算法实例

	1	2	3	4	5	6	7
x_i	A	B	C	B	D	A	B
y_j	B	D	C	A	B	A	

$c[]$

	j=0	1	2	3	4	5	6
i=0	0	0	0	0	0	0	0
1	0	0	0	0	1	1	1
2	0	1	1	1	1	2	2
3	0	1	1	2	2	2	2
4	0	1	1	2	2	3	3
5	0	1	2	2	2	3	3
6	0	1	2	2	3	3	4
7	0	1	2	2	3	4	4

$rec[]$

	j=1	2	3	4	5	6
i=1	U	U	U	LU	L	LU
2	LU	L	L	U	LU	L
3	U	U	LU	L	U	U
4	LU	U	U	U	LU	L
5	U	LU	U	U	U	U
6	U	U	U	LU	U	LU
7	LU	U	U	U	LU	U

实验3-5：最长公共子序列

- 给定两个序列X和Y，求两个序列最长公共子序列。

- 示例 1：

输入：X: "ABCADBCA" Y: "BACDBDA"

输出：

- 要求：

1 填空；

2 打印dp数组；

3 打印追踪数组rec[]。

3.5 子序列问题 最大子段和

- 问题描述：给定n个数(可以为负数)的序列 $(a_1, a_2, \dots, a_n)$ ，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。

$$\max\{0, \max_{1 \leq i \leq j \leq n} \sum_{k=i}^j a_k\}$$

- 实例1

输入：(-2, 11, -4, 13, -5, -2)

输出：最大子段和为 $a_2 + a_3 + a_4 = 20$

- 实例2

输入：[-2, 1, -3, 4, -1, 2, 1, -5, 4]

输出：6

解释：连续子数组 [4, -1, 2, 1] 的和最大，为 6。

3.5 子序列问题 最大子段和

- **算法1**: 蛮力算法, 对所有的 (i, j) 对, 顺序求和 $a_i + \dots + a_j$ 并比较出最大的和。
- **算法2**: 分治策略, 将数组分成左右两半, 分别计算左边的最大和、右边的最大和、跨边界的最大和, 然后比较其中最大者。
- **算法3**: 动态规划

3.5 子序列问题 最大子段和 算法1 蛮力

输入：数组 $A[1..n]$,

输出： $sum, first, last$

1. $sum \leftarrow 0$

2. for $i \leftarrow 1$ to n do

3. for $j \leftarrow i$ to n do

4. $thisum \leftarrow 0$

5. for $k \leftarrow i$ to j do

6. $thisum \leftarrow thisum + A[k]$

7. if $thisum > sum$

8. then $sum \leftarrow thisum$

9. $first \leftarrow i$

10. $last \leftarrow j$

和的
边界 $i-j$

找到更
大和

时间复杂度： $O(n^3)$

3.5 子序列问题 最大子段和 算法2 分治

- 算法思想:

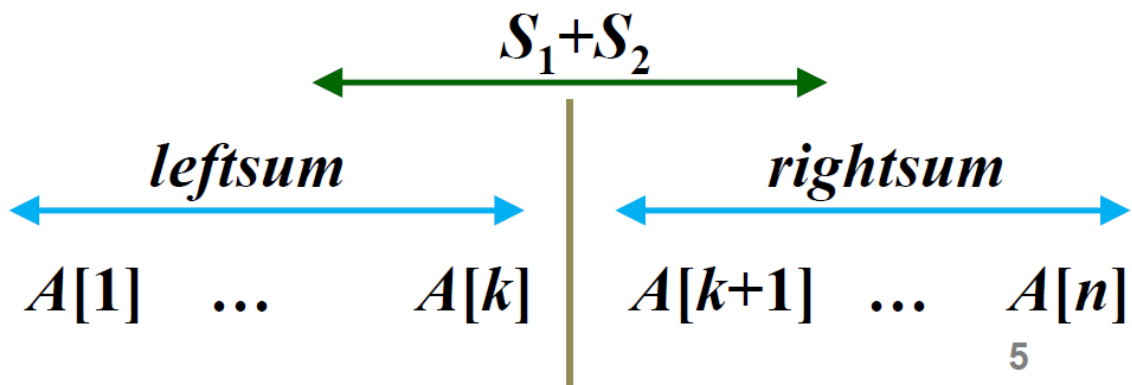
(1) 将序列分成左右两半, 中间分点center;

(2) 递归计算左段最大子段和 $leftsum$;

(3) 递归计算右段最大子段和 $rightsum$ 。

(4) 计算 $\max \{ leftsum, rightsum, S_1+S_2 \}$

其中 S_1 为 a_1 到 $center$ 的最大和 S_1 , S_2 为 $center + 1$ 到 a_n 的最大和。



3.5 子序列问题 最大子段和 算法2 分治

算法 MaxSubSum ($A, left, right$)

输入：数组 $A, left, right$ (左,右边界)

输出：最大子段和 sum 及子段边界

1. if $|A|=1$ then 输出元素(值为负输出0)
2. $center \leftarrow \lfloor (left+right)/2 \rfloor$
3. $leftsum \leftarrow \text{MaxSubSum}(A, left, center)$
4. $rightsum \leftarrow \text{MaxSubSum}(A, center+1, right)$
5. $S_1 \leftarrow A_1[center]$ //从 $center$ 向左
6. $S_2 \leftarrow A_2[center+1]$ //从 $center+1$ 向右
7. $sum \leftarrow S_1 + S_2$
8. if $leftsum > sum$ then $sum \leftarrow leftsum$
9. if $rightsum > sum$ then $sum \leftarrow rightsum$

时间复杂度

:

$$T(n) = 2T(n/2) + O(n)$$

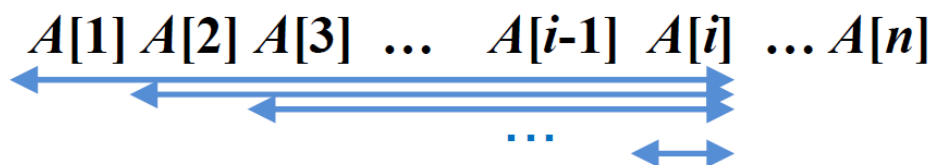
$$T(c) = O(1)$$

$$T(n) = O(n \log n)$$

3.5 子序列问题 最大子段和 算法3 动态规划

子问题界定：前边界为 1，后边界 i ，
 $C[i]$ 是 $A[1..i]$ 中必须包含元素 $A[i]$ 的
向前连续延伸的最大子段和

$$C[i] = \max_{1 \leq k \leq i} \left\{ \sum_{j=k}^i A[j] \right\}$$



最大子段和 $C[i]$

递推式：

$$C[i] = \max\{C[i-1] + A[i], A[i]\}$$

$$i = 2, \dots, n$$

$$C[1] = A[1] \quad \text{若 } A[1] > 0$$

$$C[1] = 0 \quad \text{否则}$$

$$\text{解：} \quad \text{OPT}(A) = \max_{1 \leq i \leq n} \{C[i]\}$$

3.5 子序列问题 最大子段和 算法3 动态规划

算法 **MaxSum** (A, n) **伪码**

输入：数组 A

输出：最大子段和 sum , 子段最后位置 c

1. $sum \leftarrow 0$

2. $b \leftarrow 0$

3. for $i \leftarrow 1$ to n do

子问题
后边界 i

4. if $b > 0$

5. then $b \leftarrow b + A[i]$

6. else $b \leftarrow A[i]$

7. if $b > sum$

找到更
大的和

8. then $sum \leftarrow b$

9. $c \leftarrow i$

10. return sum, c

时间复杂度： $O(n)$, 空间复杂度： $O(n)$

3.5 子序列问题 最大子段和 算法3 动态规划

1. 确定dp数组以及下标的含义

$dp[i]$: 包括下标 i 之前的最大连续子段和为 $dp[i]$ 。

2. 确定递推公式 状态转移方程

$dp[i]$ 从两个方向可以推出来:

- $dp[i - 1] + nums[i]$: 即: $nums[i]$ 加入当前连续子段和
- $nums[i]$, 即从头开始计算当前连续子段和一定是取最大的。

所以 $dp[i] = \max(dp[i - 1] + nums[i], nums[i]);$

3. dp数组如何初始化

$dp[0] = nums[0]$

4. 确定遍历顺序 从前向后遍历

5. 举例推导dp数组 $nums = [-2, 1, -3, 4, -1, 2, 1, -5, 4]$

下标:	0	1	2	3	4	5	6	7	8
$dp[i]$:	-2	1	-2	4	3	5	6	1	5

实验3-6：最大子段和

- 问题描述：给定 n 个数(可以为负数)的序列 $(a_1, a_2, \dots, a_n)$ ，找到一个具有最大和的连续子数组（子数组最少包含一个元素），返回其最大和。
- 示例 1：
输入: $[-2, 1, -3, 4, -1, 2, 1, -5, 4]$
输出: 6
解释: 连续子数组 $[4, -1, 2, 1]$ 的和最大，为 6。
- 要求：
 - 1 完成程序；
 - 2 打印dp数组；
 - 3 打印子段范围。